

VIETNAM NATIONAL UNIVERSITY OF HO CHI MINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



Web Application Development - IT093IU

Lab 8 Homework: REST API & DTO PATTERN

Nguyễn Vũ Thành Tính - ITCSIU23039

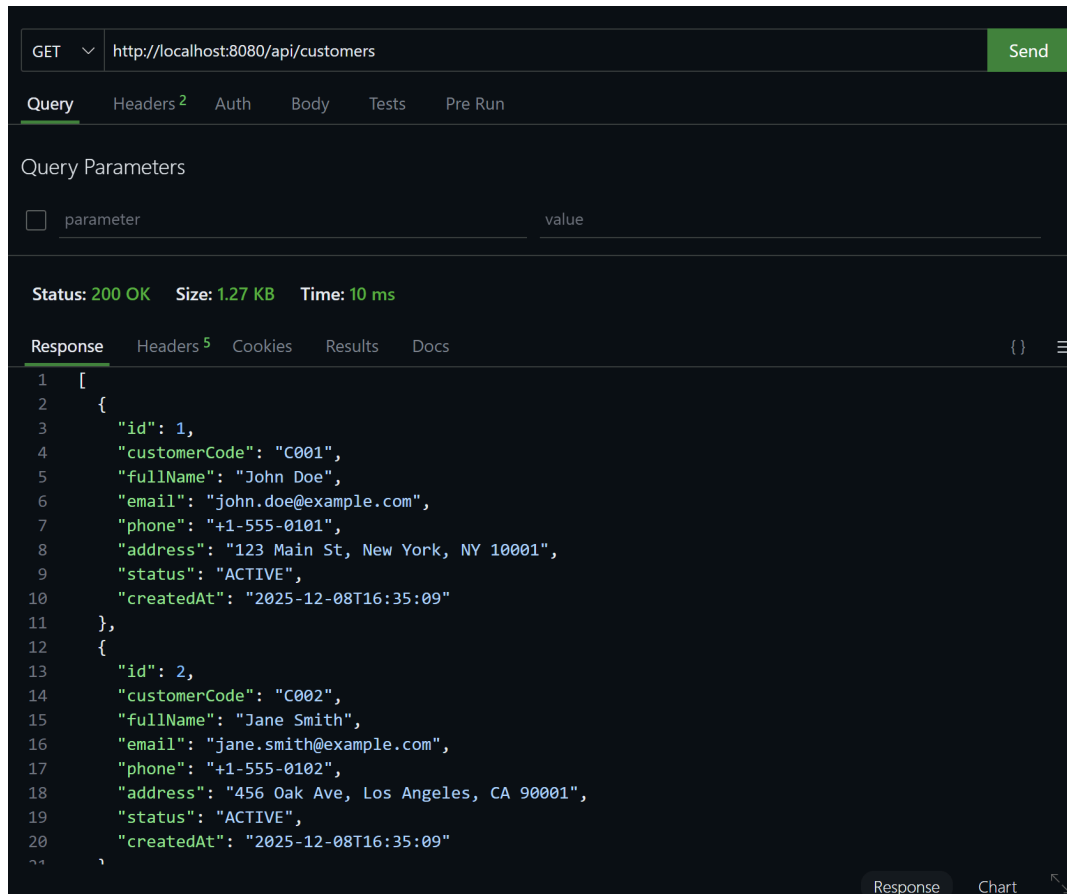
Lab Instructor: N.T.Nghia

Github link:	3
Test 1: GET All Customers	3
Test 2: GET Customer by ID	4
Test 3: POST Create Customer	5
Test 4: PUT Update Customer	6
Test 5: DELETE Customer	7
Test 6: Search Customers	8
Test 7: Validation Error	9
Test 8: Resource Not Found	10
Test 9: Duplicate Resource	11
Test 10: Search results	12
Test 11: Paginated results	13

Github link:

https://github.com/ngvuththinh/Web_Lab/tree/main/Lab_8

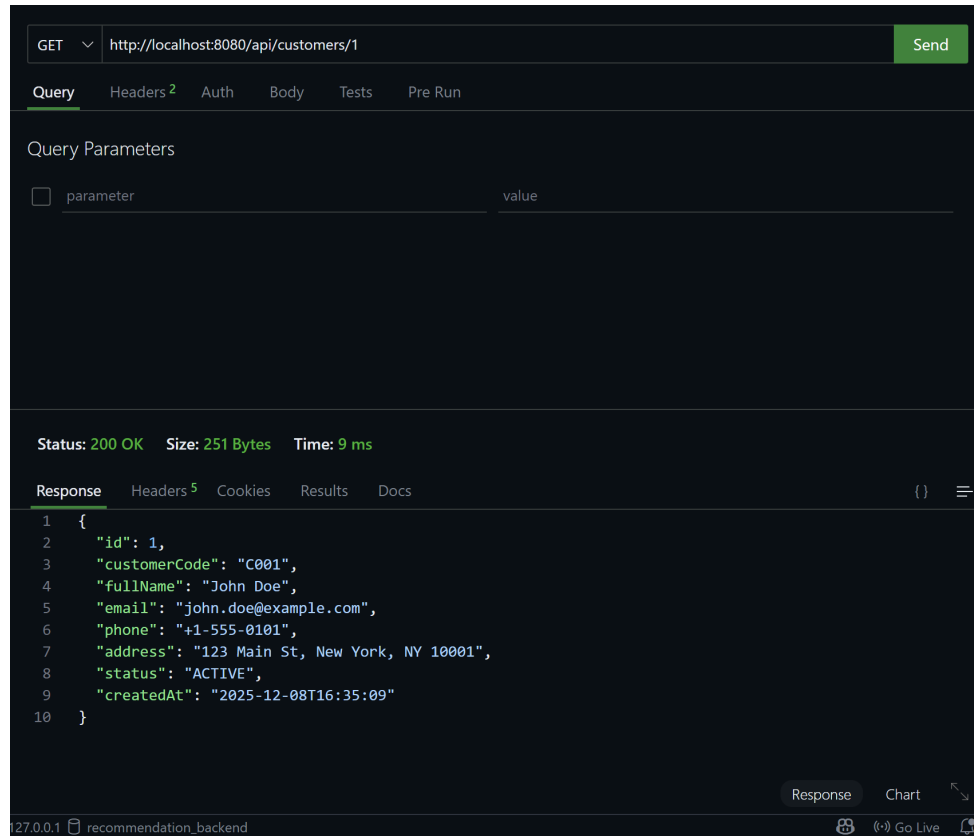
Test 1: GET All Customers



Explanation:

- **Action:** The client sends a GET request to the endpoint /api/customers.
- **Process:** The Controller receives the request and invokes `customerService.getAllCustomers()`. The Service layer calls `customerRepository.findAll()` to retrieve all customer records from the database. The entities are then mapped to DTOs.
- **Result:** The server returns **HTTP 200 OK** along with a JSON array containing the list of all customers, confirming that the retrieval logic is functioning correctly.

Test 2: GET Customer by ID



Explanation:

- **Action:** The client sends a GET request to `/api/customers/1` to retrieve a specific customer.
- **Process:** The Controller extracts the ID (1) from the URL using `@PathVariable`. The Service layer searches for the customer using `findById(1)`.
- **Result:** Since the customer with ID 1 exists in the database, the server returns **HTTP 200 OK** and the specific customer object in JSON format.

Test 3: POST Create Customer

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8080/api/customers
- Body:** JSON content:

```
1 {
2   "customerCode": "C006",
3   "fullName": "David Miller",
4   "email": "david.miller@example.com",
5   "phone": "0963149280",
6   "address": "999 Broadway, Seattle, WA 98101"
7 }
```
- Status:** 201 Created
- Size:** 266 Bytes
- Time:** 15 ms
- Response:**

```
1 {
2   "id": 7,
3   "customerCode": "C006",
4   "fullName": "David Miller",
5   "email": "david.miller@example.com",
6   "phone": "0963149280",
7   "address": "999 Broadway, Seattle, WA 98101",
8   "status": "ACTIVE",
9   "createdAt": "2025-12-08T10:25:41.1051358"
10 }
```

Explanation:

- **Action:** The client sends a POST request to `/api/customers` with a JSON body containing the new customer's details.
- **Process:**
 1. Spring Boot performs input **Validation** based on DTO annotations (e.g., checking if the email format is valid).
 2. The Service layer checks for duplicate email addresses or customer codes.
 3. If valid, the Repository saves the new entity to the database.
- **Result:** The server returns **HTTP 201 Created**. The response body contains the newly created customer data, including the auto-generated id and timestamps.

Test 4: PUT Update Customer

The screenshot displays a REST client interface with the following details:

- Method:** PUT
- URL:** http://localhost:8080/api/customers/7
- Body:** JSON content:

```
{  "customerCode": "C007",  "fullName": "David Miller",  "email": "david.millerCS@example.com",  "phone": "0963149280",  "address": "999 Broadway, Seattle, WA 98101"}
```
- Status:** 200 OK
- Size:** 260 Bytes
- Time:** 28 ms
- Response:** JSON content:

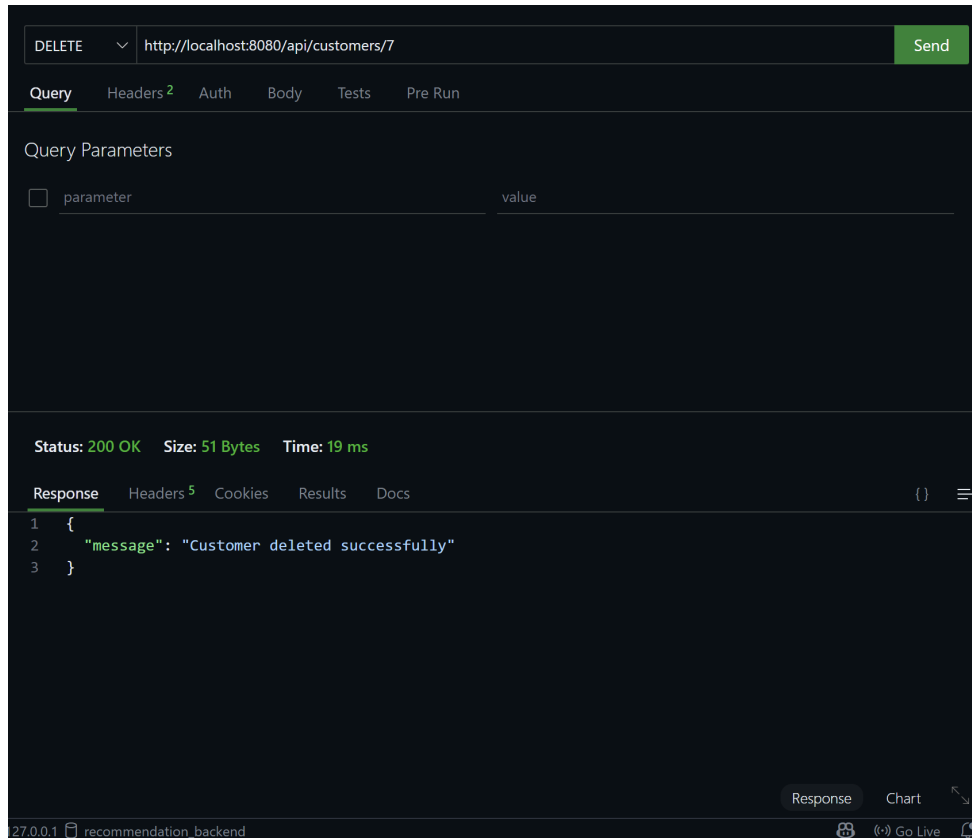
```
{  "id": 7,  "customerCode": "C006",  "fullName": "David Miller",  "email": "david.millerCS@example.com",  "phone": "0963149280",  "address": "999 Broadway, Seattle, WA 98101",  "status": "ACTIVE",  "createdAt": "2025-12-08T10:25:41"}
```

The bottom status bar shows the version 127.0.0.1 and the project name recommendation_backend.

Explanation:

- **Action:** The client sends a PUT request to /api/customers/6 to update the information for the customer with ID 6.
- **Process:** The Service verifies that the customer exists. Then, it updates the entity's fields with the data provided in the request body and saves the changes using repository.save().
- **Result:** The update is successful, and the server returns **HTTP 200 OK** with the updated customer information.

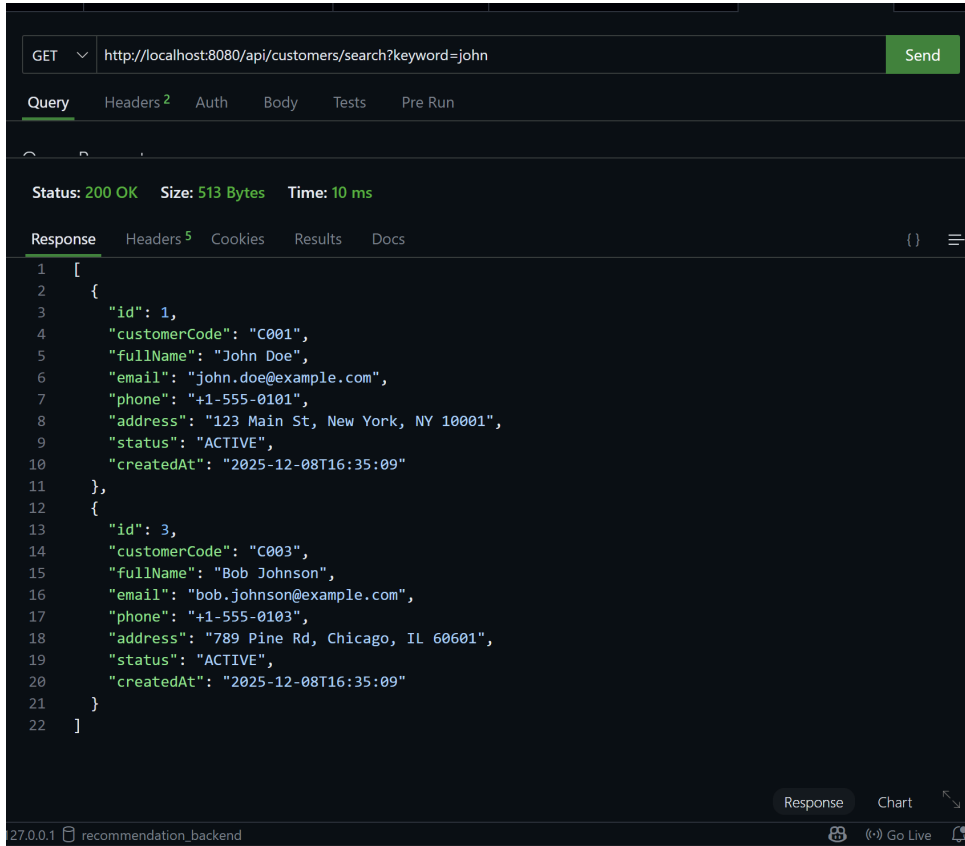
Test 5: DELETE Customer



Explanation:

- **Action:** The client sends a DELETE request to /api/customers/6.
- **Process:** The Service checks if the customer with ID 6 exists. If found, it calls repository.deleteById(6) to remove the record from the database.
- **Result:** The server returns **HTTP 200 OK** with a JSON message "Customer deleted successfully", confirming the resource has been removed.

Test 6: Search Customers



The screenshot shows a REST client interface with the following details:

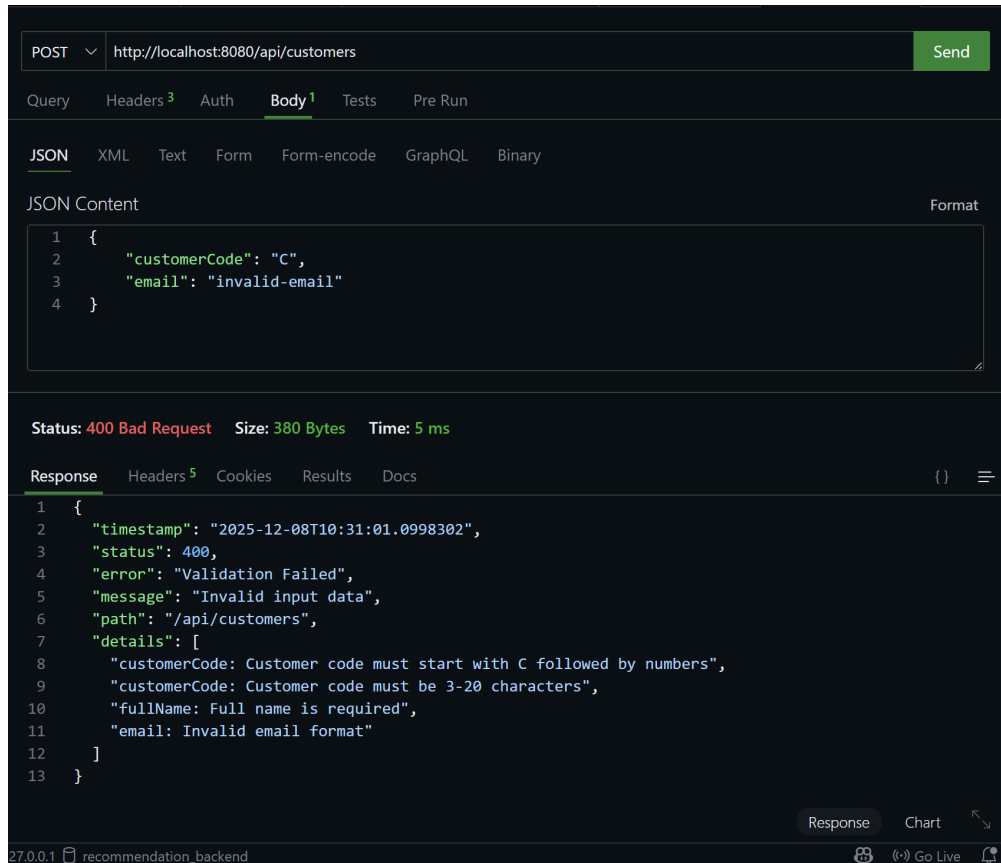
- Method:** GET
- URL:** http://localhost:8080/api/customers/search?keyword=john
- Status:** 200 OK
- Size:** 513 Bytes
- Time:** 10 ms
- Response Body (JSON):**

```
1  [
2    {
3      "id": 1,
4      "customerCode": "C001",
5      "fullName": "John Doe",
6      "email": "john.doe@example.com",
7      "phone": "+1-555-0101",
8      "address": "123 Main St, New York, NY 10001",
9      "status": "ACTIVE",
10     "createdAt": "2025-12-08T16:35:09"
11   },
12   {
13     "id": 3,
14     "customerCode": "C003",
15     "fullName": "Bob Johnson",
16     "email": "bob.johnson@example.com",
17     "phone": "+1-555-0103",
18     "address": "789 Pine Rd, Chicago, IL 60601",
19     "status": "ACTIVE",
20     "createdAt": "2025-12-08T16:35:09"
21   }
22 ]
```

Explanation:

- **Action:** The client sends a GET request to `/api/customers/search` with the query parameter `keyword=john`.
- **Process:** The Controller captures the keyword parameter (`@RequestParam`). The Repository executes a custom query (e.g., `LIKE %john%`) to find customers whose name or email matches the keyword.
- **Result:** The server returns **HTTP 200 OK** and a filtered list of customers matching the search criteria (e.g., "John Doe", "Bob Johnson").

Test 7: Validation Error



Explanation:

- **Action:** The client sends a POST request with invalid data (e.g., the customer code is too short, or the email format is incorrect).
- **Process:** The `@Valid` annotation triggers the validation logic. Since constraints are violated, a `MethodArgumentNotValidException` is thrown. The `GlobalExceptionHandler` catches this exception.
- **Result:** The request is rejected, and the server returns **HTTP 400 Bad Request**. The response body includes a detailed list of validation errors (e.g., "Invalid email format"), guiding the client to correct the input.

Test 8: Resource Not Found

The screenshot shows a REST client interface with a GET request to `http://localhost:8080/api/customers/999`. The response status is **404 Not Found**, with a size of 201 Bytes and a time of 13 ms. The response body is a JSON object:

```
1 {
2   "timestamp": "2025-12-08T10:31:29.5868806",
3   "status": 404,
4   "error": "Not Found",
5   "message": "Customer not found with id: 999",
6   "path": "/api/customers/999",
7   "details": null
8 }
```

Explanation:

- **Action:** The client attempts to retrieve a customer using a non-existent ID (e.g., 999).
- **Process:** The Service searches the database but finds no record. It then throws a custom `ResourceNotFoundException`.
- **Result:** The `GlobalExceptionHandler` handles the exception and returns **HTTP 404 Not Found** with the message "Customer not found with id: 999".

Test 9: Duplicate Resource

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/api/customers`. The request body is a JSON object with the following content:

```
1 {
2   "customerCode": "C007",
3   "fullName": "Test User",
4   "email": "john.doe@example.com"
5 }
```

The response status is **409 Conflict**, with a size of 198 Bytes and a time of 9 ms. The response body is a JSON object with the following content:

```
1 {
2   "timestamp": "2025-12-08T10:31:50.825443",
3   "status": 409,
4   "error": "Conflict",
5   "message": "Customer code already exists: C007",
6   "path": "/api/customers",
7   "details": null
8 }
```

Explanation:

- **Action:** The client attempts to create a new customer using an email address that already exists in the system (`john.doe@example.com`).
- **Process:** Before saving, the Service calls `repository.existsByEmail(...)`. Upon finding that the email is already taken, it throws a `DuplicateResourceException`.
- **Result:** The server prevents the duplicate entry and returns **HTTP 409 Conflict**, explicitly stating that the email address is already in use.

Test 10: Search results

The screenshot shows a REST client interface with a GET request to `http://localhost:8080/api/v1/customers/search?keyword=John`. The response status is 200 OK, size is 548 Bytes, and time is 37 ms. The response body is a JSON array containing two customer objects.

```
1 [
2   {
3     "id": 1,
4     "customerCode": "C001",
5     "fullName": "John Doe",
6     "email": "john.doe@example.com",
7     "phone": "0999999999",
8     "address": "123 Main St, New York, NY 10001",
9     "status": "ACTIVE",
10    "createdAt": "2025-12-10T08:55:58",
11    "links": []
12  },
13  {
14    "id": 3,
15    "customerCode": "C003",
16    "fullName": "Bob Johnson",
17    "email": "bob.johnson@example.com",
18    "phone": "+1-555-0103",
19    "address": "789 Pine Rd, Chicago, IL 60601",
20    "status": "ACTIVE",
21    "createdAt": "2025-12-10T08:55:58",
22    "links": []
23  }
24 ]
```

Explanation:

- **Action:** The client sends a GET request to `/api/v1/customers/search` with a query parameter `keyword` (e.g., "John").
- **Process:**
 1. The `CustomerRestController` captures the `keyword` using `@RequestParam`.
 2. It calls `customerService.searchCustomers(keyword)`.
 3. The `Repository` executes a custom JPQL or Derived Query (e.g., `findByFullNameContainingOrEmailContaining...`) to find customers where the Full Name, Email, or Customer Code matches the `keyword` (case-insensitive).
 4. The retrieved Entities are converted into `CustomerResponseDTO` objects.
- **Result:** The server returns HTTP 200 OK along with a JSON array containing only the customers that match the search criteria. If no matches are found, an empty list is returned.

Test 11: Paginated results

The screenshot shows a REST client interface. The request is a GET to `http://localhost:8080/api/v1/customers?page=0&size=5&sortBy=fullName`. The response is a 200 OK with a size of 1.54 KB and a time of 18 ms. The response body is a JSON object with the following structure:

```
1 {
2   "totalItems": 7,
3   "totalPages": 2,
4   "customers": [
5     {
6       "id": 4,
7       "customerCode": "C004",
8       "fullName": "Alice Brown",
9       "email": "alice.brown@example.com",
10      "phone": "+1-555-0104",
11      "address": "321 Elm St, Houston, TX 77001",
12      "status": "INACTIVE",
13      "createdAt": "2025-12-10T08:55:58",
14      "links": []
15    },
16    {
17      "id": 3,
18      "customerCode": "C003",
19      "fullName": "Bob Johnson",
20      "email": "bob.johnson@example.com",
21      "phone": "+1-555-0103",
22      "address": "789 Pine Rd, Chicago, IL 60601",
23      "status": "ACTIVE",
24      "createdAt": "2025-12-10T08:55:58",
25      "links": []
26    }
27  ]
28 }
```

Explanation:

- **Action:** The client sends a GET request to `/api/v1/customers` providing pagination parameters: `page` (page number, 0-indexed), `size` (items per page), and optionally `sortBy` (field to sort by) and `sortDir` (direction).
- **Process:**
 1. The Controller accepts these parameters and creates a `Pageable` object (specifically `PageRequest.of(page, size, sort)`).
 2. The `customerService.getAllCustomers(pageable)` is called.
 3. The Repository uses Spring Data JPA's `findAll(Pageable)` method to fetch only the requested slice of data from the database, optimizing performance.
 4. The result is a `Page<Customer>` object containing the data and metadata (total pages, total elements).
- **Result:** The server returns HTTP 200 OK. The response body is a structured JSON containing:
 - a. The list of customers for the current page.
 - b. Pagination metadata (current page number, total items, total pages), allowing the frontend to build pagination UI controls.